

FLIGHT DELAY PREDICTOR PROJECT REPORT

DATA 608 WINTER 2026 L01 GROUP 4

THANUSHA BALASUBRAMANIAN

KALISTA KOHLMAN

KATELAN NG

Table of Contents

Section 1: Problem Statement & Summary of Results	2
Section 2: Data engineering lifecycle	3
Section 2.0: Pipeline diagram	3
Section 2.1: Data generation	4
Section 2.2: Data ingestion	5
Section 2.3: Data storage	5
Section 2.4: Data transformation	6
Section 2.5: Data serving	7
Section 3: Evaluation	8
Section 4: Limitations and possible next steps	9
Data Limitations	9
Time Limitations	9
Technology and Tools Limitations	10
Next Steps	10
Section 5: Conclusions	11
Section 6: Shareable Resources	11
Section 7: Use of GenAI Tools	12
Chat GPT Usage for Historical Weather API	12
Chat GPT Usage for Amazon Athena Queries	12
Chat GPT Usage for Streamlit	12
Chat GPT Usage for Datatype Compatibility	12
References	13

Section 1: Problem Statement & Summary of Results

For this project, our aim was to predict the amount a domestic flight within the United States will be delayed based on user input. Our end goal was to create an app in which a user could receive a prediction for their flight's delay. We wanted the user to be able to input information about their flight including flight date, time, airline carrier, as well as departure and arrival airport. Our app would then provide the estimated arrival delay of the flight (in minutes) based on user input as well as the forecasted weather.

This exploration is important because flight delays can have negative impacts on passengers including inconvenience as well as potentially missed connections and the incursion of unplanned costs (Ball et al., 2010). It was estimated that flight delays had an impact of over \$30 billion to the American economy in 2022 (Travel Trends, 2023). As a result, an app that provides an estimate of how long a flight will be delayed can provide valuable information to passengers.

In the project, we successfully ingested data on US domestic flights from the Bureau of Transportation Statistics (n.d.-b) as well as Open Meteo's historical weather API (n.d.-a). We were then able to join the flight data to the weather data on flight date, time, and airport using Amazon Athena. With this joined data, we trained a machine learning model that can predict the amount, in minutes, in which a flight will be delayed. Finally, we successfully created a Streamlit app which allows users to input data about their flights and receive a prediction for their flight's delay.

Section 2: Data engineering lifecycle

Section 2.0: Pipeline diagram

The diagram in Figure 1 outlines the data pipeline and architecture we used for our project.

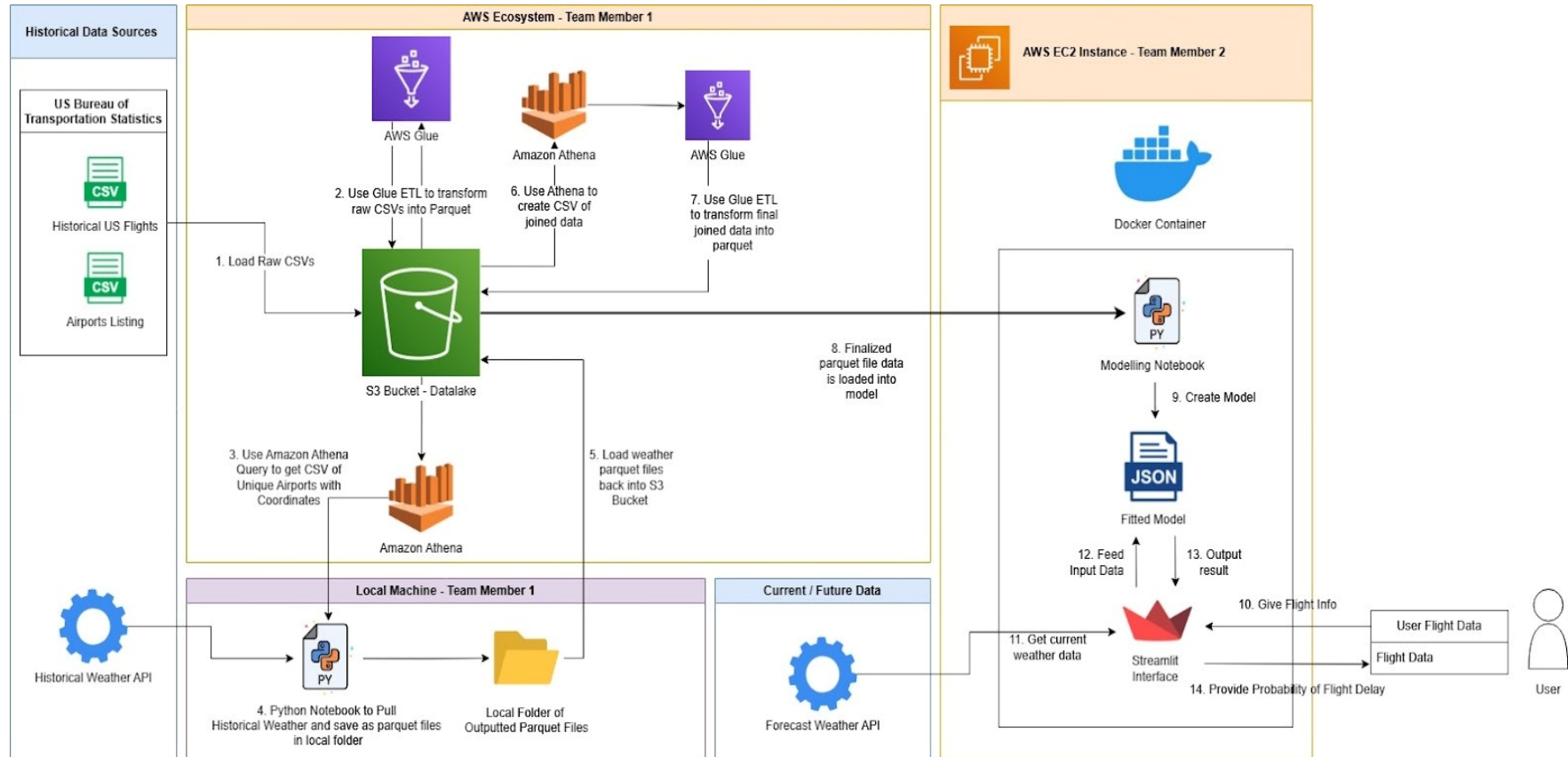


FIGURE 1: DATA PIPELINE DIAGRAM

As indicated in Figure 1, the steps in the data pipeline were as follows:

Step 1: Load flight data and airports listing CSVs into the S3 Bucket.

Step 2: Use AWS Glue to transform the flight and airports CSVs to Parquet.

Step 3: Use an Amazon Athena query to get a CSV with unique airports.

Step 4: Use a Python notebook to pull data from historical weather API and convert data to Parquet. The Python notebook requires the CSV of unique airports to pull weather based on latitude and longitude.

Step 5: Load weather Parquet files into the S3 Bucket.

Step 6: Use an Amazon Athena query to join weather data to flight data on flight date, hour, and airports.

Step 7: Use AWS Glue to convert combined flight and weather data to Parquet. Load the Parquet files to S3 Bucket.

Step 8: Load combined flights and weather data into Python notebook for modelling.

Step 9: Use the Python notebook to train a model with the combined flight and weather data. Export fitted model as JSON.

Step 10: User inputs flight information into the Streamlit app.

Step 11: App pulls weather data for origin and destination airport from forecast weather API.

Step 12: App feeds flight information input and forecasted weather to trained model.

Step 13: Model used to make a prediction for flight arrival delay.

Step 14: Streamlit app displays predicted flight delay to the user.

Section 2.1: Data generation

We primarily used data generated from two sources in our project. The first data source was the “On-Time: Reporting Carrier On-Time Performance (1987-present)” dataset provided by the United States Bureau of Transportation Statistics (n.d.-b). The dataset included information about scheduled and actual flight departure and arrival times for domestic flights in the United States. The dataset includes flight information from all US carriers representing at least 0.5% of domestic air travel passenger revenue. US carriers are required to report this information for the flights they operate (Bureau of Transportation Statistics, n.d.). Data in the Bureau of Transportation Statistics portal is reported in monthly tables and is updated regularly. The most recent available data is from November 2025.

This dataset covers flights between 1987 to 2025. However, we only examined data from 2022 to 2025. As the Covid-19 pandemic constituted a major disruption to the aviation industry (Sun et al., 2022), we believed that these years are the most relevant to predicting whether current flights will be delayed. Data we used included flight date, airline carrier, route distance, arrival delay, scheduled departure and arrival times, as well as origin and destination airport.

The second data source we used is Open-Meteo’s historical weather API (n.d.-a). Open-Meteo provides historical weather data dating from 1940. Open-Meteo collects its data from partnerships with national weather services. Data for the historical weather API is archived from Open-Meteo’s weather forecast API and is updated daily. We used the API to access weather information from 2022 to 2025 for each of the airports

covered in the flight dataset. Data we used included hourly information for temperature, precipitation, and wind speed.

Section 2.2: Data ingestion

The flight data from the United States Bureau of Transportation Statistics is provided on their website as monthly CSVs (n.d.-b). As a result, we downloaded a separate CSV for each of the 36 months from November 2022 to October 2025. In total, these CSVs were about 3.5 GB in size with over 20 million rows. We then manually loaded the downloaded CSVs into an S3 Bucket using the “upload files” functionality in AWS.

To acquire all the required weather data, we first needed a list of all the unique airports covered in the flight data. To get this list, we performed a query on the data using Amazon Athena. In total, we had 366 unique airports.

To pull data from Open Meteo’s historical weather API, we needed the latitude and longitude of each airport. To get this information, we used a Bureau of Transportation Statistics dataset which includes location and time zone information about each airport (n.d.-a). We joined this table to the list of unique airports. This allowed us to get the latitude and longitude for each airport in the flight data.

Next, we used the latitudes and longitudes to pull data from the historical weather API using a Python notebook. The Python notebook was run on a local machine. For each airport location, the Python notebook sent an API request for the three year time period covered by our flight data, extracted the hourly variables returned by the API, and constructed a Pandas dataframe containing the weather measurements along with both UTC and local timestamps. The processed data was then written to uncompressed Parquet files. We chose to write the data to Parquet files as they would require less storage than CSV.

The Python notebook API ingestion script was designed to operate sequentially and included a two second delay between requests to prevent API throttling. The script also recorded the outcome of each request in a batch log and wrote the weather data for each airport to a separate Parquet file. Although storing one file per airport increased the number of output files, and was less efficient for storage, it was better for the nature of our task. By having a separate Parquet file for each airport, if only a few airports in a batch failed to load, the entire batch did not have to be reprocessed. The script skipped over any airports we already had Parquet files for. This ensured that each successive run of the script would pull new airports. Additionally, by manually changing the batch setting, we were able to work on a few batches, stop when we hit the hourly API call limit, and return to the task later.

The pulled weather data included data for all 366 airports for every hour between November 2022 and October 2025. As a result, the data had over 9 million rows in total. After the weather data was pulled, it was loaded into the S3 Bucket.

Section 2.3: Data storage

We stored all data for the project in an S3 Bucket. We selected an S3 Bucket for storage as opposed to an Elastic Block Store because we wanted a storage solution that scaled to fit the size of our data. If we had opted for an Elastic Block Store, we would have paid for any unused storage room. In contrast, with an S3 bucket, we only paid for the storage that we used. This was an important consideration due to our limited AWS academy credits.

Additionally, with the S3 bucket there is the option to make files accessible to the public via an access control list and setting objects in the bucket to public. By allowing the finalized Parquet files of joined data to be public,

we were able to allow other teammates to have other portions of the workflow on their AWS environments, while still having access to the necessary data. This spread out the cost of AWS resources, so that we did not need to use only one person's Learner Lab credits.

We stored both our flight data and weather data as Parquet files. We chose this format because Parquet files provide efficient storage. In other words, data saved as Parquet requires less storage space than the same data stored as CSV. Additionally, Parquet files enable fast queries with Amazon Athena (Databricks Staff, n.d.).

Section 2.4: Data transformation

Data transformation is the portion of the data engineering lifecycle where our project deviated the most from our proposal. In our proposal, we suggested using an AWS Glue Crawler to transform our CSV files into Parquet. This suggestion was based on our understanding of Amazon Glue at the time. When we started working with the data and tools, we learnt that Amazon Glue Crawler does not actually transform data. Instead, it populates the AWS Glue Data Catalog which Amazon Athena uses to infer schema for its queries.

When we tried using the AWS Glue Crawler on our flight CSVs, we were able to get the correct number of rows when we ran our queries, but the data in the rows was incorrect. The data in the results were offset on any columns where a string contained a comma. We attempted to troubleshoot the Glue Crawler but could not get a working fix. Instead, we opted to use the Glue Visual ETL tool. The Glue Visual ETL allows users to configure a data ETL job through a drag and drop interface. When we used the visual ETL, we were able to actually transform the data from CSVs into Parquet format. There was no issue with offsetting as the Glue Visual ETL parsed the CSVs correctly and had an option to not separate columns with commas if they were wrapped in quotations marks (ie. it would not split string fields that had commas inside them).

We used the Glue Crawler to get the schema for the weather Parquet files. We did not have the same issue with the Parquet files that we did with the CSVs, as Parquet files are stored with a schema. As a result, the Crawler was able to use the predefined schema rather than having to infer a schema based on commas as delimiters, as it does for CSV files.

In addition to Amazon Glue, we also used Amazon Athena for data transformation. We used Amazon Athena to write a query which joined weather data to the flight data. We did two separate joins: one for weather at the origin airport and one for weather at the destination airport. Both joins were based on date, airport, and scheduled hour of departure/ arrival. The query took five minutes to run and the results were output to a CSV file in the S3 bucket. We used a left join in the query because we wanted to get the corresponding weather for every flight and not get weather that did not match a flight. Using Amazon Athena to run the query was efficient as it took only five minutes to run despite creating an output of over 20 million rows of data.

After we had a CSV file with the results, we again used the visual ETL to transform the results into Parquet files. We converted to Parquet because we wanted to be able to run Amazon Athena queries on the resulting files to confirm there was no duplication of rows after joining the data. We matched the number of rows in the joined data to the number of rows in the flight data to confirm that no flights were duplicated. After we had completed the ETL to get the query results in Parquet, we found there was a way to write a query in Athena which would output the results directly as Parquet. This would involve writing a "Create a Table as Select" statement. We did not implement this into the workflow of the project, as further steps in our workflow depended on our original Parquet files. Specifically, changing this step could have affected the model and final app.

Section 2.5: Data serving

While our project primarily focused on the data engineering lifecycle, we developed a machine learning model and front-end application to give practical use to the processed data generated from previous steps. The application demonstrates that the data is properly structured and production-ready, enabling the practical usage of our engineered work. The success of our application relies on the usage of our engineered data as well as the speed and accessibility of data retrieval. Our primary measure of success for data serving was whether the application can receive input and produce an output within a reasonable time which we defined as less than one minute.

To create our model, we loaded the joined flight and weather data into a Dask dataframe inside a Python notebook. Next, we performed several preprocessing steps to clean the data, such as data type conversions and further filtering. Once the dataframe was in a suitable format, we split the dataset into predictor features and the target variable, arrival delay.

We selected XGRegressor as our machine learning model for its strong performance in handling large amounts of mixed numerical and categorical features (Brownlee, 2024). The model was trained on the historical data and evaluated on its mean absolute error as our performance metric. The mean absolute error was relatively high at 25.3 minutes. This means that the average prediction was 25.3 minutes off from the actual arrival delay. After training, we preserved the fitted model as a JSON file so that we would not need to retrain the model every time a user used the Streamlit app.

The use of the Dask dataframe was an important decision in the feasibility of training the model. We decided to use Dask rather than Pandas due to the size of our dataset, which had more than 20 million rows. Dask allows for parallel and distributed computing which significantly reduces memory requirements. To estimate the memory savings from using Dask, we measured peak memory usage during training in a high RAM environment. With Dask, the peak memory usage during training was approximately 13.6 GB. As a comparison, we conducted the same model training with a Pandas dataframe. With Pandas, the peak memory usage during training was approximately 28.3 GB. As a result, the use of Dask allowed us to use far less computational resources.

Our front-end application was developed using Streamlit to create a user-friendly graphical interface. The application allows the end user to input information known about their flight, such as departure and destination details, and receive a summary of their flight details and estimated arrival delay from the prediction model in return. The streamlit.py script serves four main purposes:

1. Receive basic user input.
2. Compute/extract relevant data from user input. This lessens the level of cognitive demand placed on the user.
3. Process user input into a compatible format for our model.
4. Output flight details in a clear and informative format.

Our app conducts several steps to process user input. For location information, our interface only requires the user to select the origin and destination airport codes. As a result, our app extracts additional details such as longitude and latitude from the list of unique airports stored in the S3 bucket. The coordinates are then used to determine the distance between the two airports using the GeoPy package. The coordinates are also used to pull the relevant weather data from Open-Meteo's forecasted weather API (n.d.-b). The app stores the extracted data as a Dask dataframe in a format that is usable by our predictive model. The app then feeds this dataframe into our model for prediction.

For output, the application displays numerous details regarding the user's flight, forecasted weather conditions at the origin and destination airports, as well as the predicted arrival delay in minutes. This information is served in a dashboard-like design, making the output easily digestible for the user.

We deployed an EC2 instance to host and store all our modelling and streaming applications. The use of Docker allowed us to containerize the application environment. This approach makes our application accessible, reproducible, and significantly simplifies deployment. With the configuration of our Dockerfile and EC2 security group, our app can be made accessible by exposing a public IP to end users.

Section 3: Evaluation

In our proposal, we outlined several measures of success for the project. Our ultimate measure of success was whether we were able to create an app that could take user input and provide a prediction for flight arrival delay with a response time of less than one minute. To ensure that we were able to achieve this goal, we created three intermediate measures of success. These intermediate measures as well as their outcomes are listed below.

- 1. All flight data retained upon conversion to Parquet:** After using Amazon Glue to convert the flight data in CSV to Parquet, we reconciled the record count to ensure completeness. During reconciliation, we found that row counts in the CSV and Parquet files matched for each month. As a result, we considered this measure to be successful.
- 2. Each flight record joined to corresponding weather data:** Upon joining flight and weather data, every flight origin and destination was matched to the corresponding weather data with no data missing. We also found that the number of rows after the join matched the number of rows for the original flights data. Additionally, we ensured that there was no duplication of flights.
- 3. Model's ability to predict flight arrival delays:** Our model is able to provide a prediction for flight arrival delay in minutes. However, the mean absolute error in predictions is 25.3 minutes. This means that the average prediction is 25.3 minutes off from the actual value. Therefore, the model has low predictive power. However, our focus on this project was our data pipeline, not modelling, so we decided not to pursue optimizing the model. It is also important to note that modelling had significant memory demands with a peak RAM usage of 13.6 GB. As a result of the low predictive power of the model and significant memory usage, we considered our project to only be moderately successful on this measure.
- 4. Run time:** In our initial proposal, we did not include goals for how long specific processes should take for run time. We did not include these goals because many of the AWS tools we used (Glue Studio Visual ETL, Glue Crawler, Amazon Athena) were new to us. As a result, we had no reference point for how long these tools should take to run. Although this was not an initially included measure, we do consider this as a success. The query that took the longest to run was the join query which appended weather data to the flight data. This query took approximately 5 minutes to run. The Glue Visual ETLs and the Glue Crawler each took 2 to 3 minutes to run. This is impressive, as a query on a traditional SQL database which returns 20+ million records typically takes much longer than 5 minutes to run.

In our original proposal, we used the group's ability to complete tasks in our workflow on or before the scheduled due date as an additional measure of success. By breaking down the project requirements on a granular level, we were able to know exactly what needed to be completed, when it needed to be completed

by, and by whom. This schedule allowed us to better manage the project to ensure we stayed on task. As a group, we did meet our task deadlines, but we often had a minimal viable product to proceed to the next task in the workflow rather than an optimal product. Due to this lack of optimization, we sometimes went back to re-iterate and perform completed tasks in a more optimal way. This allowed us to still move onto subsequent tasks in the project while implementing improvements to completed tasks. One example of this re-iteration is that even after we had the weather data, we went back and changed how we pulled the data to be more efficient. Despite this process of re-iteration, we were still able to complete tasks by the scheduled deadline which allowed us to move forward with project. As a result, we considered the project to be successful on this measure.

As stated above, our ultimate goal for the project was to create an app that could accept user input and provide a timely prediction for the arrival delay of their flight. Our app is, indeed, capable of taking input from a user and providing such a prediction. Moreover, our app can provide a response within a few seconds, far less than our stated goal of one minute. Our app also provides additional information to the user including the distance of their flight as well as the forecasted weather for both the origin and destination airports. As a result, we consider our project to be successful in its stated overall goal.

Section 4: Limitations and possible next steps

While we consider our project to be successful in its stated goal, there are three primary areas in which our project has limitations. These areas are data limitations, time limitations, as well as technology and tool limitations.

Data Limitations

As our project relied on the data sourced by the Bureau of Transportation Statistics and Open-Meteo, we faced three limitations due to the lack of reported information available to us. First, as we limited our scope to ingest flight data from the Bureau of Transportation Statistics, our model was only trained on data from US domestic flights. As a result, our app cannot provide predictions for international flights or domestic flights in countries outside the United States.

Second, our model only considered the weather at the origin and destination airports. It did not consider the weather along the flight route. As a result, our model cannot consider that there may be weather along the flight route that could cause a delay. We could not include weather data along the route because our flight data did not include route information. Considering enough locations along the route would have also greatly increased the scale of our data.

Lastly, the app cannot provide predictions for flights more than three days in advance. Our app has this limitation due to its use of a weather API to get the forecasted weather for the origin and destination airport which only allows us to pull weather information up to three days in the future.

Time Limitations

Due to the short, three-week time frame to complete this project, we had to limit the scope for the project. One way we decided to narrow the scope was in not tuning the hyperparameters of the model. Our model has a high mean squared error. This means that the model's predictions for flight delays are far from the actual value. As a result, even though our app can provide the user with a prediction for the arrival delay of their flight, this prediction has low accuracy. Due to time constraints and the primary focus of the project being on

data engineering, we considered the untuned model to be acceptable. However, if we had additional time for tuning, we may have been able to improve our modeling predictions.

An additional way in which the time constraint limited us is that when we found improved ways to perform tasks, we did not have the time to implement these improvements in our workflow. For example, when we found a way to have Amazon Athena output query results directly to Parquet using a “Create Table As Select” (CTAS) statement, we could not implement this improvement as it would affect downstream analytics. If a potential improvement was found too late in the process (after subsequent steps reliant on it were completed), we did not have the time to go back and implement it.

Technology and Tools Limitations

Our project’s last area of limitation is in the constraints posed by the tools and technologies we used. For example, we ran into memory constraints during the training of the model. Our EC2 instance was initially configured with 8 GB of RAM and 100 GB of storage. In our first attempt to train the model, the memory was insufficient because it could not handle the large volume of data and quickly timed out. As a result, we allocated an additional 8 GB of memory to the instance. Although this was a feasible fix for our data, this was a short-term solution for our memory problem. In the future, if we wanted to scale the project to cover additional data, we would likely run into memory difficulties. Acquiring large amounts of memory is expensive, so this could present a major challenge to scalability.

Another limitation in our project was that the Open-Meteo Historical Weather API we used was free and, therefore, had significant limitations on calls. The API had limits for individual calls, as well as per minute and per hour limits. Although we were able to find a workaround (as described in the Data Transformation section), the workaround is not scalable. If we were to try to get historical weather for more airports, or for longer periods of time, the data ingestion scripts may be infeasible to implement due to run time.

Next Steps

Potential next steps for this project could be taken to address these limitations. These next steps could include the inclusion of international flights data as well as domestic flight data from outside the United States. This would allow our app to be able to provide predictions for flights other than US domestic flights. We could also explore including a larger date range for our training data. It is possible that this extra data would allow us to build a model with stronger predictive power. Additionally, we could invest more time and computing resources to tune the hyperparameters of the XGRegressor model. XGRegressor models require significant tuning to achieve optimal performance (Brownlee, 2024). As a result, it is highly likely that we would be able to create a model with greater predictive power by taking the time and resources to tune hyperparameters. However, it is important to note that all of these potential steps would require significantly more memory resources than we used for this project. Therefore, prior to scaling the scope of this project, we would need to optimize our code to be as resource efficient as possible.

For an additional next step, we could also put a greater focus on security. During our project we did not focus on this area. For example, we used an S3 Bucket with ACL enabled for public access and set some folders to be accessible the public via URL. For future work, we would invest more time in configuring security settings, so we were able to restrict access to only team members working on the project. As we grow in our experience with AWS, we hope to learn more about security and data privacy settings. For this project, all the data we gathered was publicly available, so we felt that the reduced security measures were acceptable. In a scenario with private data, we would need to investigate ways of restricting access while making data available to all teammates.

Section 5: Conclusions

Our ultimate goal for the project was to create an app in which a user could input information about their flight and receive a predicted value for their flight's arrival delay, in minutes. To accomplish this goal, we ingested data generated by the Bureau of Transportation Statistics (n.d.-b) and Open Meteo's historical weather API (n.d.-a). We then transformed the data by joining the flight data to the weather data using Amazon Athena. All data was stored in Parquet format in a S3 Bucket. The transformed data was used to train a machine learning model. Data was then served to the user through a Streamlit app which allows the user to input their flight details and receive a prediction for their flight's delay which is calculated through the use of the predictive model. Our project was ultimately successful in its goal in creating an app that can provide timely responses to user input.

Through the completion of the project, we gained a better understanding of the data engineering life cycle. This learning included a better understanding of data engineering tools such as APIs, AWS Glue, and Amazon Athena. We also learnt about techniques for serving data with Streamlit.

Another fundamental learning outcome of this project was the importance of the efficient use of computational resources such as storage and memory. A key takeaway was being aware of budget constraints while working with large scale data under limited resources. While working in AWS, we had to be conscious of the limitations of the Learner Account tier throughout the project. Key techniques for preserving budget we learnt included using tools such as Parquet files and S3 Buckets to allow for efficient storage and the use of Dask to reduce memory use.

Additionally, we learned how to learn new tools and technologies in a less structured manner than how we learn in class. We learned in a very hands-on way how to use completely new technologies that we had never been taught in a formal setting. When something did not go according to plan or a tool did not work the way we thought it would, we pivoted and troubleshooted so that we could still make progress. It was a lot of work, but at the end, when we had a working product, it was very personally rewarding.

Potential next steps for the project include the incorporation of additional data from international flights and domestic flights from outside the United States. The inclusion of this data would allow our app to make predictions for a broader range of flights. However, it is important to note this expansion of data would require significantly greater memory resources than what was used for this project. This could be a motivation for further research into optimizing our created scripts to be more resource conscious.

Section 6: Shareable Resources

Link to our repository:

https://github.com/kngtm1/Flight_Delay_Prediction

Materials available in the repository include:

- [guide to configuring AWS settings](#) with instructions for setting up the S3 Bucket and performing the AWS processes up to Step 7 converting joined data to Parquet
- [Athena queries](#) folder with all queries used in Amazon Athena
- [Open Meteo Airport weather](#) Python script with historical weather API calls
- [Colab folder](#) with Python notebooks for modelling and the Streamlit app
- [README.md](#) with instructions for app deployment methods via EC2 and Colab Notebook

Additionally, we also have a [video](#) demonstrating the use of the app. The video is only accessible to University of Calgary e-mail account holders and will expire on May 31, 2026.

Section 7: Use of GenAI Tools

Chat GPT Usage for Historical Weather API

When we made the historical weather data ingestion notebook, ChatGPT was used to support design decisions, code structuring, and debugging. ChatGPT was not used to generate the final script in its entirety. Instead, it provided guidance and example code snippets that were adapted and modified throughout the process of creating the script to call the weather API.

For example, ChatGPT suggested grouping multiple airports into a single API request to reduce the number of calls, but the final script instead processes airports individually to improve reliability and simplify error handling. This way if part of a batch failed, we would still be able to use the portions of the batch that passed. And we would be able to easily identify which airports passed and which failed in a batch.

ChatGPT's recommendation to manually adjust timestamps using the API's UTC offset was replaced with an approach that converts timestamps to the airport's local time zone and stores them as formatted local date-time strings. The retry behavior and batch size were also adjusted to avoid long retry loops and API throttling, resulting in a workflow that processes airports in batches of 50, writes one parquet file per airport, and logs batch results to allow the process to resume safely if interruptions occur.

Chat GPT Usage for Amazon Athena Queries

ChatGPT was used to draft some of the more complex queries. All queries were reviewed for input parameters and filters used, as well as joins. Additionally, the output of the joining query was reconciled to the source data by reconciling the count of monthly records to the parquet files (which had previously been reconciled to the CSV files). We also spot checked the weather for a few records.

Chat GPT Usage for Streamlit

ChatGPT provided assistance in giving suggestions for the types of visual elements to use for our graphical interface. We were able to prototype an initial framework when we prompted for an example of a front-end interface for an application that provides flight details. With this, we were able to build on this for our final product.

Chat GPT Usage for Datatype Compatibility

ChatGPT verified and corrected the compatibility of the data types between the parameters received by the model and the input parameters of the user. The usage of AI helped us understand the underlying data structures and the encoding and metadata of categorical features in the Dask package.

References

- Ball, M., Barnhart, C., Dresner, M., Hansen, M., Neels, K., Odoni, A. R., Peterson, E., Sherry, L., Trani, A., & Zou, B. (2010, October 1). *Total delay impact study: A comprehensive assessment of the costs and impacts of flight delay in the United States*. Repository and Open Science Access Portal. <https://rosap.ntl.bts.gov/view/dot/6234>
- Brownlee, J. (2024). *XGBoost advantages and disadvantages (pros vs cons)*. XGBoosting. <https://xgboosting.com/xgboost-advantages-and-disadvantages-pros-vs-cons/>
- Bureau of Transportation Statistics. (n.d.-a). *Aviation support tables: Master coordinate* [Data set]. United States Department of Transportation. https://www.transtats.bts.gov/DL_SelectFields.aspx?gnoyr VQ=FLL&QO fu146 anzr=N8vn6v10+f722146+gnoyr5
- Bureau of Transportation Statistics. (n.d.-b). *On-time: Reporting carrier on-time performance (1987-present)* [Data set]. United States Department of Transportation. https://www.transtats.bts.gov/DL_SelectFields.aspx?gnoyr VQ=FGJ&QO fu146 anzr=b0-gvzr
- Databricks Staff. (n.d.). *What is Parquet?* Databricks. <https://www.databricks.com/blog/what-is-parquet>
- Open-Meteo. (n.d.-a). *Historical weather API* [API]. <https://open-meteo.com/en/docs/historical-weather-api>
- Open-Meteo. (n.d.-b). *Weather forecast API* [API]. <https://open-meteo.com/en/docs>
- Travel Trends. (2023, September 26). *Flight disruption: The impact on the economy and environment*. International Travel & Health Insurance Journal. <https://www.itij.com/latest/news/flight-disruption-impact-economy-and-environment#author-1831>